

Краткая справка по решению задачи Р5 "И свершится праведная месть - краткая справка по решению"

Постановка.

Даны три бинарные матрицы $A, B, C \in \{0, 1\}^{n \times n}$, $1 \leq n \leq 4000$. Нужно проверить равенство

$$AB = C$$

в поле $\mathbb{F}_2$ (сложение по модулю 2, умножение как по обычным битам).

Матрицы задаются компактно: каждая матрица описывается одной строкой, содержит n блоков по $\lceil n/4 \rceil$ шестнадцатеричных цифр. Каждый блок задаёт одну строку матрицы: из блока берутся биты в порядке от старших к младшим, лишние биты в конце обрезаются до длины n .

Идея алгоритма.

Прямое перемножение за $O(n^3)$ не подходит для $n \leq 4000$, поэтому используется *вероятностный алгоритм Фрейвальдса*.

Основная идея: выбрать случайный вектор $r \in \{0, 1\}^n$ и проверить равенство

$$A(Br) = Cr$$

в $\mathbb{F}_2$.

Обозначим $D = AB - C$ (в $\mathbb{F}_2$ это просто $D = AB \oplus C$). Если $AB = C$, то $D = 0$, и для любого r выполнено $Dr = 0$. Если $AB \neq C$, то $D \neq 0$, и для случайного r

$$\Pr[Dr = 0] \leq \frac{1}{2}.$$

Поэтому одна проверка может ложно вернуть «равны» с вероятностью не более $1/2$. Повторяя проверку k раз с независимо выбранными r , получаем вероятность ошибки не более 2^{-k} .

Схема алгоритма.

1. Считать n и три матрицы A, B, C из шестнадцатеричного ввода.
2. Хранить каждую матрицу как массив битовых строк:

$$A[i] \in \{0, 1\}^n, \quad i = 0, \dots, n-1,$$

удобно использовать `std::bitset<4000>`.

3. Повторить k раз (например, $k = 25$):
 - (a) Сгенерировать случайный ненулевой вектор $r \in \{0, 1\}^n$.
 - (b) Вычислить $t = Br$, затем $u = At$.
 - (c) Вычислить $v = Cr$.
 - (d) Если $u \neq v$, вывести NO и завершить.
4. Если все проверки прошли успешно, вывести YES.

Реализация операций. Все умножения и сложения выполняются в $\mathbb{F}_2$:

- скалярное произведение строки на вектор:

$$\langle \text{row}, r \rangle = \bigoplus_{j=0}^{n-1} (\text{row}_j \wedge r_j),$$

то есть чётность количества позиций, где оба бита равны 1;

- на практике: `tmp = row & r`, затем `tmp.count() \& 1`.

Таким же образом реализуется умножение матрицы на вектор.

Для воспроизводимости используется фиксированное значение `seed` (например, `srand(123456)`), без инициализации от текущего времени.

Оценка сложности.

- Время одной проверки: две операции «матрица $\times$ вектор» и одна — для Cr , каждая за $O(n^2)$ битовых операций. Итого $O(n^2)$.
- Время всего алгоритма: $O(kn^2)$. При $n \leq 4000$ и умеренном k (например, $k = 25$) укладывается в лимит по времени.
- Память: хранение трёх матриц по n^2 бит: $3n^2$ бит $\approx 3 \cdot 16 \cdot 10^6$ бит ≈ 6 МБ при $n = 4000$, что значительно меньше лимита 256 МБ.

Свойства.

- Если $AB = C$, алгоритм *всегда* выводит YES.
- Если $AB \neq C$, вероятность вывода YES не превышает 2^{-k} .

Таким образом, алгоритм является односторонне вероятностным (Monte Carlo) с очень малой вероятностью ошибки при разумном числе повторений.